

Solutions to Week 9 Conceptual Questions

1

Refer lecture slides.

2

Refer lecture slides.

3

Consider the pseudocode of **append** function:

```
1 /*
2  Dynamic Array structure:
3  - A.array      stores the array of elements
4  - A.capacity   stores the current capacity (initially 0)
5  - A.nElems     stores the number of elements (initially 0)
6  - A.Free()     frees the array allocation
7
8  Allocate(c) returns a new DynamicArray with capacity c and nElems = 0
9  Copy(A,A') copies all elements of A' into A -- assumes A.capacity >= A'.nElems
10 */
11
12 append(DynamicArray A, elem e) {
13   // Resize if array is full
14   if (A.nElems == A.capacity) {
15     // Double capacity or initialize to 1 if empty
16     newA = Allocate(max(1, 2 * A.capacity));
17     // Copy old elements to new array
18     Copy(newA, A);
19     newA.nElems = A.nElems;
20     A.Free();    // free old array
21     A = newA;    // update reference
22   }
23   // Append the new element
24   pos = A.nElems;
25   A.array[pos] = e;
26   A.nElems += 1;
27 }
```

3.1 Amortized analysis using the aggregate method

τ_i denotes the true cost of performing the i -th **append** operation in a sequence of n appends. From the pseudocode, we have:

$$\tau_i = \begin{cases} O(i), & \text{if } i-1 \text{ is a power of 2 (and resizing occurs),} \\ O(1), & \text{otherwise.} \end{cases}$$

Starting from a zero capacity array, for a sequence of n append operations, there are $m = \lfloor \log_2 n \rfloor + 1$ array-resizing events (running lines 14-26 in the pseudocode on page 1) where the capacity changes from $0 \rightarrow 2^0 \rightarrow 2^1 \rightarrow 2^2 \rightarrow \dots \rightarrow 2^{m-1}$, with 2^{m-1} being the largest power of $2 \leq n$. The cost of append associated with resize is $O(i) = O(2^j)$ effort, where $0 \leq j \leq m-1$.

The remaining $n-m$ operations each require $O(1)$ effort (ignoring lines 14-22 and running lines 24-26 in the pseudocode).

Up to constant factors, the total cost of n appends can be characterized as:

$$\begin{aligned} \sum_{i=1}^n \tau_i &= n - m + \underbrace{\sum_{j=0}^{m-1} 2^j}_{=2^m-1} \\ &= n - m + 2^m - 1 \\ &\leq n + 2^m \quad (\because m \geq 0) \\ &\leq n + 2n \quad (\because 2^{m-1} \leq n \implies 2^m \leq 2n) \\ &\leq 3n. \end{aligned}$$

This gives the upper bound on the true cost of a sequence of n **append** operations as $O(n)$, making the per-operation amortized cost of $O(1)$.

3.2 Amortized Analysis Using the Accounting Method

In this method of analysis, one coin of credit represents a unit of constant work. Assume that λ coins are charged for each **append** operation, sufficient to support any sequence of **append** operations.

- One coin pays for the immediate cost of inserting the new element e into $A.\text{array}[\text{pos}]$ (line 25 of the pseudocode).
- One additional coin is stored as credit on the newly inserted element at that index pos . This credit will pay for copying (line 18) that element during the next array-doubling event.
- One further coin is stored as credit on the element that *mirrors* this position in the lower half of the array, specifically at index $\text{pos} - \frac{A.\text{capacity}}{2}$. This additional credit is necessary because elements in the lower half of the array in any state were already copied during the previous doubling event, which consumed the credit originally stored on them when they were first appended. The newly assigned credit therefore prepays their next copy operation during the next doubling event.

Thus, charging 3 coins at the time an element is inserted into the array (line 25) ensures that any sequence of **append** operations can always be completed, since

- one coin pays for the immediate insertion cost,
- one coin prepays the future copying cost of the newly appended element (during the next doubling event), and
- one coin replenishes the credit of a corresponding element in the lower half of the array whose previous credit has already been spent during the previous doubling event.

Therefore, copying of each element into a new doubled array during a doubling event has already been prepaid. Hence charging $\lambda = 3$ coins per **append** operation is sufficient, implying that the amortized cost of **append** is $O(1)$.

3.3 Amortized analysis using the potential method

Let:

- $A.\text{nElems}_{i-1}$ be the number of elements in the dynamic array before the i -th element was appended.
- $A.\text{nElems}_i$ be the number of elements in the dynamic array after the i -th element is appended.
- $A.\text{capacity}_{i-1}$ be the capacity of the dynamic array before the i -th element was appended.
- $A.\text{capacity}_i$ be the capacity of the dynamic array after the i -th element is appended.

The specified potential function Φ in terms of the variables used in the pseudocode (on page 1) is:

$$\Phi = 2 \times A.\text{nElems} - A.\text{capacity}.$$

Consider the amortized cost a_i of the i -th append in terms of the potential difference when the array A goes from $A_{i-1} \rightarrow A_i$ state. If τ_i is the true cost of the i -th append, we have the amortized cost of this operation defined as:

$$a_i = \tau_i + \Phi(A_i) - \Phi(A_{i-1})$$

We have to analyze the amortized cost in two distinct scenarios:

1. When $i - 1$ is not a power of two, which means that there is capacity in the array's current state to append a new element into it. Here $\tau_i = 1$ (because it is $O(1)$ effort).
2. When $i - 1$ is a power of two requiring array-doubling before the i -th element is appended into the doubled array. Here $\tau_i = i$, or symbolically in terms of the array variables $\tau_i = A.\text{nElems}_i = A.\text{capacity}_{i-1} + 1$.

When $i - 1$ is not a power of 2

$$\begin{aligned} a_i &= \tau_i + \Phi(A_i) - \Phi(A_{i-1}) \\ &= 1 + 2 \times \text{A.nElems}_i - \text{A.capacity}_i - 2 \times \text{A.nElems}_{i-1} + \text{A.capacity}_{i-1}. \end{aligned}$$

But, in this scenario we have the following identities:

- $\text{A.nElems}_i = \text{A.nElems}_{i-1} + 1$, and
- $\text{A.capacity}_i = \text{A.capacity}_{i-1}$.

Therefore:

$$\begin{aligned} a_i &= 1 + 2 \times \text{A.nElems}_i - \text{A.capacity}_i - 2 \times \text{A.nElems}_{i-1} + \text{A.capacity}_{i-1} \\ &= 1 + 2 \times \text{A.nElems}_{i-1} + 2 - \text{A.capacity}_{i-1} - 2 \times \text{A.nElems}_{i-1} + \text{A.capacity}_{i-1} \\ &= 1 + 2 = 3. \end{aligned}$$

When $i - 1$ is a power of 2

$$a_i = \tau_i + \Phi(A_i) - \Phi(A_{i-1})$$

Mainly, in this scenarios we have the following identities:

- $\tau_i = \text{A.nElems}_i$
- $\text{A.nElems}_i = \text{A.capacity}_{i-1} + 1$
- $\text{A.capacity}_i = 2 \times \text{A.capacity}_{i-1}$
- $\text{A.nElems}_{i-1} = \text{A.capacity}_{i-1}$

Therefore,

$$\begin{aligned} a_i &= \text{A.nElems}_i + 2 \times \text{A.nElems}_i - \text{A.capacity}_i - 2 \times \text{A.nElems}_{i-1} + \text{A.capacity}_{i-1} \\ &= \text{A.capacity}_{i-1} + 1 + 2 \times \text{A.capacity}_{i-1} + 2 - 2 \times \text{A.capacity}_{i-1} - 2 \times \text{A.capacity}_{i-1} + \text{A.capacity}_{i-1} \\ &= 1 + 2 = 3. \end{aligned}$$

This provides the amortized cost of $O(1)$ across both the scenarios of `append`.